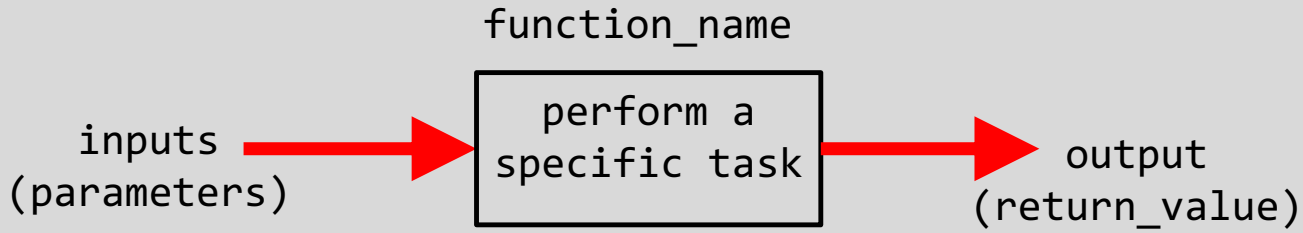
EEET2601 Engineering Computing 1

More about Functions

Quang Tran

RMIT UNIVERSITY

Function declaration




```
<return_type> function_name(parameters) {
    // code to perform a specific task
    return return_value;
}
```

function header

function body

Function design

- When designing a function, we must think carefully about how to design the **inputs** and **outputs** of the function
- Four options:
  1. No inputs and no outputs
  2. No inputs but one output
  3. Inputs but no outputs
  4. Inputs and one output

Example

Design a function to determine if an integer is even

Example: No inputs and no outputs

```c
#include <stdio.h>

// Declare a function
void get_int_check_even_display();

int main(int argc, char const *argv[]) {
    get_int_check_even_display();       // Call the function

    return 0;
}

// Define the function
void get_int_check_even_display() {
    int n;

    // Input a positive integer
    printf("Enter a positive integer: ");
    scanf("%d", &n);

    // Check if the given integer is even and display accordingly
    if (n % 2 == 0) {
        printf("%d is even\n", n);
    } else {
        printf("%d is odd\n", n);
    }
}
```

Example: No inputs but one output

```c
#include <stdio.h>

// Declare a function
int get_int();

int main(int argc, char const *argv[]) {
    int num = get_int();            // Call the function

    // Check if the given integer is even and display accordingly
    if (num % 2 == 0) {
        printf("%d is even\n", num);
    } else {
        printf("%d is odd\n", num);
    }

    return 0;
}

// Define the function
int get_int() {
    int n;

    // Input a positive integer
    printf("Enter a positive integer: ");
    scanf("%d", &n);

    return n;
}
```

Example: Inputs but no outputs

```c
#include <stdio.h>

// Declare a function
void check_even_and_display(int num);

int main(int argc, char const *argv[]) {
    int num;

    // Input a positive integer
    printf("Enter a positive integer: ");
    scanf("%d", &num);

    check_even_and_display(num);        // Call the function

    return 0;
}

// Define the function
void check_even_and_display(int n) {
    // Check if the given integer is even and output accordingly
    if (n % 2 == 0) {
        printf("%d is even\n", n);
    } else {
        printf("%d is odd\n", n);
    }
}
```

Example: Inputs and one output

```c
#include <stdio.h>

// Declare a function
int check_even(int n);

int main(int argc, char const *argv[]) {
    int num;

    printf("Enter a positive integer: ");
    scanf("%d", &num);

    int even = check_even(num);         // Call the function
    if (even == 1) {
        printf("%d is even\n", num);
    } else {
        printf("%d is odd\n", num);
    }

    return 0;
}

// Define the function
int check_even(int n) {
    if (n % 2 == 0) {
        return 1;                   // 1 represents TRUE
    } else {
        return 0;                   // 0 represents FALSE
    }
}
```

Questions

Which version in the examples is the best?

Why?

Is there anything can be improved in that version?

Function design (cont'd)

- There are two options to pass data to a function
  1. Pass by value: pass the value of the data to the function
  2. Pass by reference: pass the pointer of the data to the function
- Terminologies
  - Formal parameters: the parameters used in function definition
  - Actual parameters: the parameters used in function call
  - Arguments: common names for both formal and actual parameters

Pass by value

- The value of a variable is passed to the function, i.e. the value is copied to the formal parameter in the function
- Different memory is allocated to the actual and formal parameter
- The value of the actual parameter cannot be modified by changes to the formal parameter

Example: Pass by value

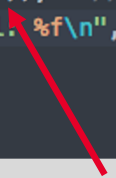
```c
#include <stdio.h>

// Function definition
float square(float num) {
    num = num * num;     // Calculate square of num and store it in num
    return num;          // Return the result
}

int main(int argc, char const *argv[]) {
    float f = 7.2;
    printf("f before the function call: %f\n", f);
    printf("square(f): %f\n", square(f));   // Call by value
    printf("f after  the function call: %f\n", f);

    return 0;
}
```

```
f before the function call: 7.200000
square(f): 51.839996
f after  the function call: 7.200000
```



The value of f is copied into the formal parameter num inside square(), i.e. same as num = 7.2

Pass by reference

- The address of a variable (pointer) is passed to the function
- Same memory is used for both the actual and the formal parameter since only address is used by both
- The value of the actual parameter can be modified by the formal parameter

Example: Pass by reference

```c
#include <stdio.h>

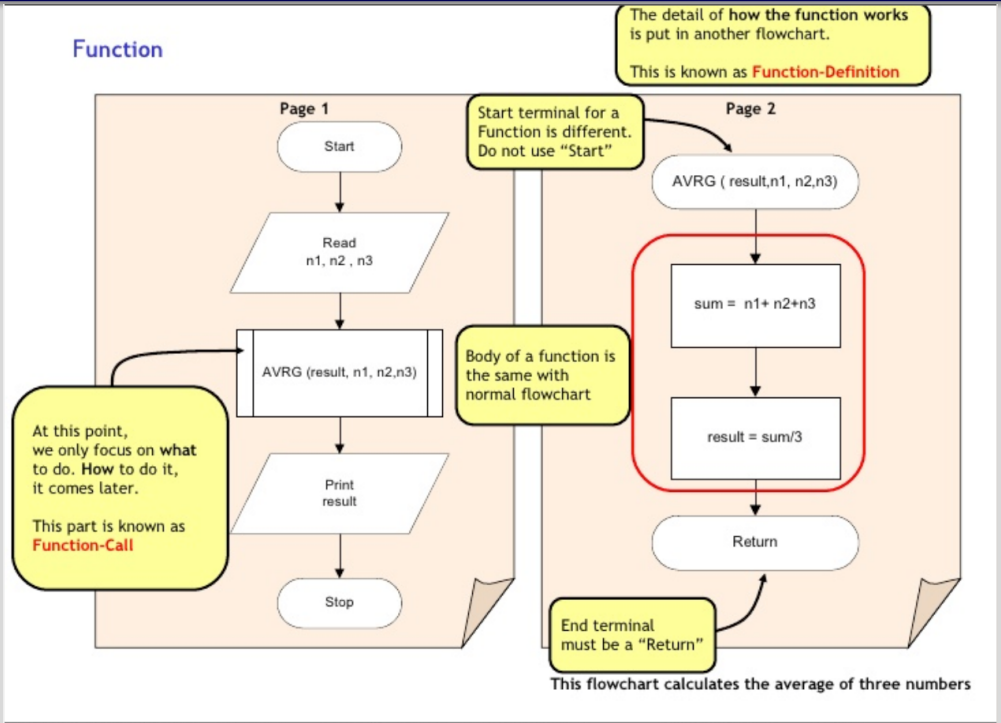
// Function definition
void square(float *p) {
    *p = (*p) * (*p);    // Calculate and update *p
}

int main(int argc, char const *argv[]) {
    float f = 7.2;
    printf("f before the function call: %f\n", f);
    square(&f);     // Call by reference by passing the address of f
    printf("f after  the function call: %f\n", f);

    return 0;
}
```

```
f before the function call: 7.200000
f after  the function call: 51.839996
```

How to draw a function using flowchart?



Nesting of functions

- C allows nesting of functions, i.e. to calling one function inside another function's body

```
function1() {
   // function1 body
   function2();
   // function1 body
}
```

- However, if `function2()` also calls `function1()` inside its body, it will lead to infinite nesting and the program will never terminate

Recursion

- Recursion is a special kind of nesting function when a function call itself in the function body

```
function1() {
    // function1 body
    function1();
    // function1 body
}
```

- To avoid infinite nesting, the recursive function must have a base case to converge

Recursion: Example

factorial.c

```c
#include <stdio.h>

int factorial(int n) {
    return (n == 0)? 1: n * factorial(n - 1);
}

int main(int argc, char const *argv[]) {
    int num;
    printf("Enter a positive integer: ");
    scanf("%d", &num);
    printf("%d! is %d\n", num, factorial(num));

    return 0;
}
```

Recursion: Tower of Hanoi problem

https://www.youtube.com/watch?time_continue=31&v=YstLjLCGmgg

The recursion takes lots of space when call → over head of the stack

mathematical induction

Recursion

- Recursion can give you very elegant solutions for several challenge problems
- But a recursion solution could be less efficient in term of running time compare to an iterative solution

References

1. Fresh2Refresh, C Programming Tutorial, 2020.
2. StudyTonight, C Programming Language Tutorial, 2020.
3. Geeksforgeeks, Tower of Hanoi, 2020.